L7-v5

2026年10月19日 10:49

两个功能

维度	上下文压缩	Anthropic 协议
解决的问题	上下文会满，满了请求会失败	端口①只有一个实现，对比验证
改动性质	新增模块	扩展已有模块
判据	引入了一个需要独立演进的概念（有状态、有策略、L10 依赖）	不引入新概念，只是把一个接口的第二个实现补上
与循环的关系	协作者（循环调用它）	被循环调用（provider_.stream）
关键纪律	切点铁律 + 锚点 + 失败降级	适配器归一化 + 协议无关逻辑不复制

文件清单

文件	变化类型	核心新增
compaction.hpp	新增文件	压缩器契约：五层级联、摘要器端口、切点
compaction.cpp	新增文件	五层级联实现、两种摘要器、抖动保护
types.hpp	修改	① 新增 CompactionEvent；② 修正 forLayer(L5_History) 语义；③ 新增 UTF-8 安全截断工具（K-21）；④ Turn 新增 compactions 字段
agent_loop.hpp	只加不改	setCompactor() / compactionEvents() / compactIfNeeded()
agent_loop.cpp	只加不改	在闸门之后、组装之前插入 compactIfNeeded()；压缩用量计入 spentTokens_
context_budget.hpp	修改	把 estimateTokens / fitTo / extractSignatures / extractSymbolNames 从 private 提到 public（压缩器也要用）
context_budget.cpp	补全实现	① 补上三个"声明了但没定义"的函数；② assemble() 真正使用预算和降级链；③ measure() 修正：tool_call 参数也要算 token
providers.cpp	重大修改	① 新增 AnthropicProvider（第二个协议）；② StreamSink 抽象；③ 修 I-8（重试判据）；④ 吸收三家结构性差异
tool_registry.cpp	小修	maybePersistLargeResult 的截断改为 utf8Prefix（K-21）
main.cpp	重大修改	① 支持 AF_PROVIDER=anthropic；② AF_CONTEXT_WINDOW；③ AF_COMPACT = 1lm/local/off；④ 新增 I-7 ~ I-32 自检；⑤ 压缩可观测性输出

上下文压缩

第一层："级联"而不是"一刀切"

直觉1：上下文满了就截断。

> 截断会产出"半个函数"、"半条 SQL"、"只剩左括号的条件表达式"——这种内容对模型的伤害比"什么都没有"更大，因为它看着像完整的，模型会基于残缺代码做出错误判断，而且它不会意识到自己看到的是残片。

直觉2：用 LLM 写摘要。

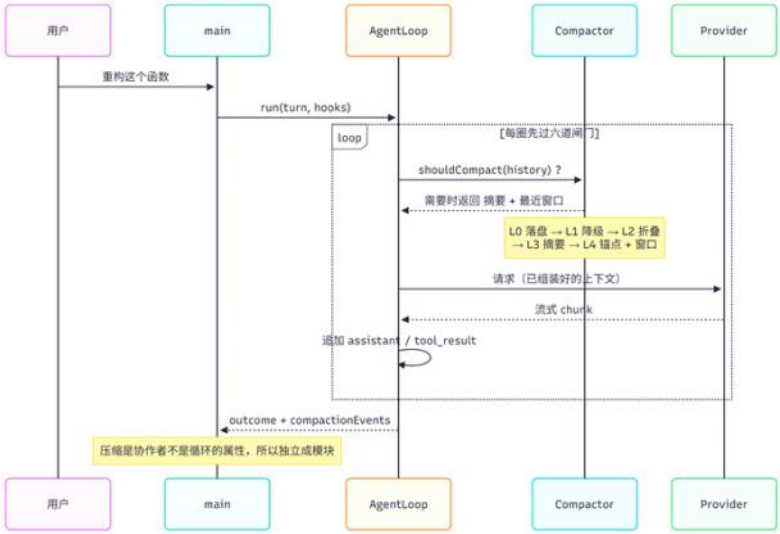
> 压缩本身就贵。LLM 摘要那一次调用，输入恰恰就是想压掉的那段上下文——为了省 40k token，先花掉 30k token 去生成摘要。如果每次都直接走 L3，那么"上下文满了"本身会变成越压越贵。

正解：先用免费的手段，不够再花钱——"级联"。

第二层：五层级联的结构

级	手段	成本	位置	处理逻辑
L0	大结果落盘	≈ 0	tool_registry（不在 compaction.cpp）	60K 字符的工具结果 → 只留 2KB 预览 + 路径
L1	单条消息降级链	≈ 0	compaction.cpp	一条超大消息走 Full→Signatures→SymbolNames→PathOnly
L2	折叠老的工具结果	≈ 0	compaction.cpp	老工具结果压成"前 300 字符 + 说明"
L3	一串历史 → 交接摘要	低/高	compaction.cpp	摘要器可换：本地抽取 / LLM
L4	只留锚点 + 最近窗口	≈ 0	compaction.cpp	丢弃老消息

```
if (历史 > 预算) {
    L1: 单条降级        → 还超?
    L2: 折叠老工具结果 → 还超?
    L3: 摘要            → 还超?   (失败则跳过，不炸)
    L4: 丢弃            → 还超?  → thrashing 报告
}
```



agent_loop 的接线

```
class AgentLoop {
public:
    // 构造函数**一个字没动** — 加"可选依赖"的正确方式是 setter，不是第 5 个参数。
    AgentLoop(IModelProvider& provider, ToolRegistry& registry,
              ContextAssembler& assembler, LoopLimits limits = {});

    LoopOutcome run(Turn& turn, const LoopHooks& hooks = {});
    int   spentTokens() const noexcept;
    Usage totalUsage() const noexcept;

    // ---- ★ L7 新增（只加不改）----
    void setCompactor(std::shared_ptr<Compactor> c);
    const std::vector<CompactionEvent>& compactionEvents() const noexcept;

private:
    void compactIfNeeded(Turn& turn, const LoopHooks& hooks);   // ★ L7 新增
    // ...
    std::shared_ptr<Compactor>   compactor_;      // ★ L7 新增
    std::vector<CompactionEvent> compactionEvents_; // ★ L7 新增
};
```

agent_loop.cpp

```
for (int step = 0; step < limits_.maxSteps; ++step) {
    // 闸门 6 / 3 / 2 前置检查...

    // ---- ★ L7：上下文压缩（在**组装之前**，不是在组装里面）-------------
    //  ★ 位置有两个讲究：
    //    ① 在闸门之后 — 已经该停的时候不要再花钱去做摘要
    //    ② 在组装之前 — 压缩改的是"历史本身"，组装只负责把历史摆好。
    //       把压缩塞进 assemble() 会有一个隐藏的后果：摘要那次模型调用
    //       发生在**组装器内部**，循环看不见它 → 账单少算一笔 →
    //       **闸门 2 会撒谎**。所以压缩必须由拥有 provider_ 的循环来驱动。
    compactIfNeeded(turn, hooks);

    // ---- A. 组装上下文 ----
    ChatRequest req = assembler_.assemble(turn);
    // ...
}
```

```
void AgentLoop::compactIfNeeded(Turn& turn, const LoopHooks& hooks) {
    if (!compactor_) return;

    const int histTokens = Compactor::estimateHistoryTokens(turn.history);
    if (!compactor_->shouldCompact(histTokens)) return;

    CompactionOutcome out = compactor_->compact(turn.history, histTokens,
                                                std::nullopt, std::nullopt, turn.input);

    // ---- 记账：摘要也是钱 ----------------------------------------------
    spentTokens_ += out.summaryUsage.total();
    total_.inputTokens      += out.summaryUsage.inputTokens;
    // ...

    // ---- 观测：把每一次压缩报出去 --------------------------------------
    for (const auto& e : out.events) {
        compactionEvents_.push_back(e);
        if (hooks.onNotice) hooks.onNotice("上下文压缩 " + e.detail + ...);
    }
}
```

压缩流程：整体调度

```
compact(history, 150000)
│
├─ 1. shouldCompact(150000)?
│     150000 > 10000 → true，继续
│
├─ 2. 算锚点
│     history[0].role == User → anchorEnd = 1
│      (history[0] 永远不压缩)
│
├─ 3. 找切点 cut = findCutPoint(history, 1, 4000)
│      (从尾部往前累加 4000 token，再往前挪到合法位置)
│     假设得到 cut = 80
│     → prefixEnd = 80
│     → 老历史 = history[1..80)，最近窗口 = history[80..102]
│
├─ 4. protectedEnd = 1 (初始只保护锚点)
│
├─ 5. L1: 对 history[1..80) 做单条降级
│     如果还超预算，继续
│
├─ 6. L2: 对 history[1..80) 里的 tool 消息做折叠
│     如果还超预算，继续
│
├─ 7. L3: 把 history[1..80) 摘要成一条消息
│     成功 → protectedEnd = 2 (锚点 + 摘要都不许删)
│     失败 → 跳过，继续 L4
│     如果还超预算，继续
│
├─ 8. L4: 删掉 history[2..keepFrom) (保留锚点/摘要 + 最近窗口)
│
└─ 9. 抖动检查: 如果压完还超预算，attempts_++
```

```
第一步：从尾往前累加**全部**消息，找到 boundary
  acc = 0
  boundary = 102
  for i from 101 down to 1:
      acc += estMessage(history[i])
      boundary = i
      if acc >= 4000: break

  假设累加到 i=80 时 acc 到 4000 → boundary = 80

第二步：从 boundary 往前挪到最近的合法切点
  for i from 80 down to 1:
      if history[i].role != Tool:   // isLegalCutPoint
          return i

  如果 history[80] 是 tool，继续往前到 79、78... 直到非 tool
  假设 history[78] 是 assistant → return 78
```

```
if 工具是 Read 工具 (persistThresholdChars == SIZE_MAX) :
    直接返回 (豁免，否则"读文件→落盘→再读落盘文件"死循环)
if content.size() <= persistThresholdChars:
    直接返回
写文件 .af-dump/toolout-{时间戳}-{工具名}.txt
content = utf8Prefix(content, 2000) + "已落盘到 {路径}，用 Read 打开"
```

```
ToolResult maybePersistLargeResult(ToolResult r, const ToolDef& def,
                                   const std::string& dumpDir) {
    if (def.persistThresholdChars == SIZE_MAX) return r;            // 豁免
    if (r.content.size() <= def.persistThresholdChars) return r;

    std::error_code ec;
    fs::create_directories(dumpDir, ec);

    const auto now = std::chrono::steady_clock::now().time_since_epoch().count();
    const std::string name =
        "toolout-" + std::to_string(now) + "-" + def.name + ".txt";
    const fs::path path = fs::path(dumpDir) / name;

    std::ofstream ofs(path, std::ios::binary | std::ios::trunc);
    if (!ofs) {
        r.content = utf8Prefix(r.content, 2'000) +
                    "\n\n[... 输出过大且落盘失败，以上为前 2000 字符 ...]";
        return r;
    }
    ofs.write(r.content.data(), std::streamsize(r.content.size()));
    ofs.close();

    const std::size_t total = r.content.size();
    r.content = utf8Prefix(r.content, 2'000) +
                "\n\n[... 共 " + std::to_string(total) +
                " 字符，已落盘到 " + path.string() +
                "，如需完整内容请用 Read 打开该文件 ...]";
    r.persisted     = true;
    r.persistedPath = path.string();
    return r;
}
```

L1：单条消息降级

输入：history[1..80) 里所有消息
输出：超大的单条消息被替换成降级后的内容

```
cap = budget / perMessageCapDiv     // 默认 10000 / 2 = 5000 token
for i in [1, 80):
    if estimateToken(history[i].content) <= cap:
        continue     // 没超，不动
    history[i].content = fitTo(history[i].content, cap, applied)
```

```
输入: content (比如 3000 行代码) , cap (5000 token ≈ 20000 字符)
|
├─ 试 Full: content 本身 <= cap?
|    是 → 返回 content, applied=Full
|
├─ 试 Signatures: extractSignatures(content)
|    每一行只要含 '(' 或以 class/struct/def/func/export 开头就保留
|    比如:
|      "int func_1(int a, int b) {"   ← 保留
|      "    return a + b;  // 实现细节"  ← 丢弃
|      "}"                             ← 丢弃
|    如果结果 <= cap → 返回, applied=Signatures
|
├─ 试 SymbolNames: extractSymbolNames(content)
|    在 Signatures 基础上, 砍掉 '(' 之后的部分
|    "int func_1(int a, int b) {" → "int func_1"
|    如果结果 <= cap → 返回, applied=SymbolNames
|
├─ 试 PathOnly: content 的第一行
|    content.substr(0, content.find('\n'))
|    如果结果 <= cap → 返回, applied=PathOnly
|
└─ 都放不下 → 返回空串, applied=Drop
```

L2：折叠老工具结果

输入：history[1..80) 里的 tool 消息

输出：超长的 tool 结果被替换成"前 300 字符 + 说明"

```
for i in [1, 80):
    if history[i].role != Tool:
        continue
    if history[i].content.size() <= foldThresholdChars(1200):
        continue
    total = history[i].content.size()
    preview = utf8Prefix(history[i].content, 300)    // 按 UTF-8 边界截
    history[i].content =
        "[早前工具结果已折叠] 原始 " + total + " 字符 → 仅保留前 300 字符\n"
        + preview
        + "\n[... 已折叠。需要完整内容请用 Read 重新读取 — "
          "路径在对应的工具调用参数里, 那份参数没有丢 ...]"
```

L3：摘要

输入：history[1..80) 整段老历史（约 40 条消息）
输出：一条摘要消息，插在锚点之后

```
1. 收集 toSummarize = history[1..80)
2. 构造 SummarizeRequest
3. 调 summarizer_->summarize(req) → SummaryResult
   - 成功 → 重建历史: [锚点] + [摘要] + [最近窗口]
   - 失败 → 跳过 L3, 继续 L4 (不抛异常)
4. protectedEnd = 2 (摘要也不许再被 L4 删)
```

L4：丢弃

输入：压完 L1/L2/L3 还超预算

输出：删掉老历史中间的一段

```
1. 算受保护前缀的 token 占用
   headTokens = sum(estimateToken(history[i])) for i in [0, protectedEnd)

2. 剩余预算 = budget - headTokens
   keepFrom = findTailStartByBudget(history, protectedEnd, 剩余预算)
   // 从尾部往前累加, 直到放不下为止, 再往前挪到合法切点
   // 至少保住最后一条消息

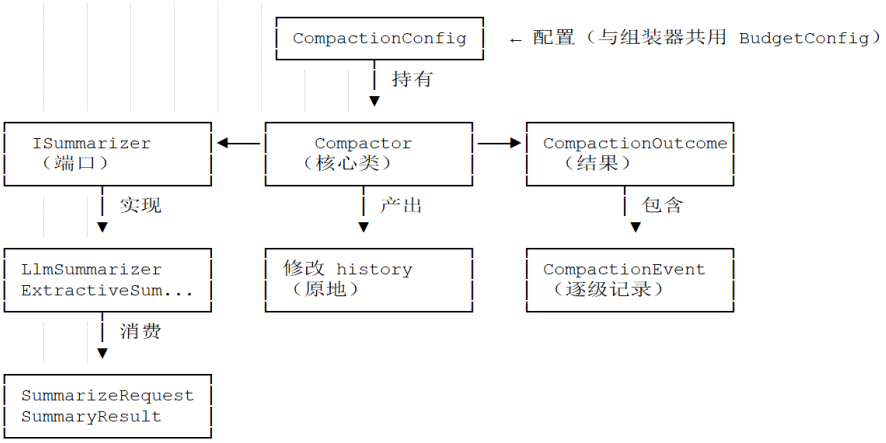
3. 删除 [protectedEnd, keepFrom)
   history.erase(protectedEnd, keepFrom)
```

```
budget = 10000 - headTokens(300) = 9700

从 history[102] 往前:
  acc = est(history[102]) = 500    → best = 102
  acc += est(history[101]) = 800   → best = 101
  ...
  累加到 9700 时 acc 会超过 → break

返回 best (最近的合法切点)
```

Compactor类型关系



分层归类

层	类型	作用
配置层	CompactionConfig、BudgetConfig	决定"压到多少、用什么策略"
端口层	ISummarizer、SummarizeRequest、SummaryResult	摘要的可换实现
实现层	LlmSummarizer、ExtractiveSummarizer	两种摘要策略
核心层	Compactor	五层级联的主逻辑
结果层	CompactionOutcome、CompactionEvent	逐级记录，供 UI / 测试

数据结构

```cpp
// ① 配置：与组装器共用同一份 BudgetConfig
struct CompactionConfig {
    bool          enabled = true;
    BudgetConfig  budget;             // ★ 与组装器共用同一份
    int  keepRecentTokens  = 20'000;  // 最近多少 token 逐字保留
    int  foldThresholdChars = 1'200;  // 单条工具结果超过它就折叠
    int  perMessageCapDiv  = 2;       // L1 的单条上限 = 历史预算 / 这个数
    int  maxSummaryChars   = 4'000;   // 摘要自身也要有上限
    int  maxAttempts       = 3;       // 抖动保护
};

// ② 结果：逐级记录，供 UI / 测试
struct CompactionOutcome {
    bool triggered = false;
    bool thrashing = false;
    std::vector<CompactionEvent> events;   // ★ 逐级记录
    Usage  summaryUsage;                   // ★ 必须计入成本（闸门 2）
    int tokensBefore = 0, tokensAfter = 0;
    int keptMessages = 0, droppedMessages = 0;
};

// ③ 事件：每一次压缩的可观测记录（在 types.hpp）
struct CompactionEvent {
    int level = 0;              // 0..4
    int tokensBefore = 0;
    int tokensAfter  = 0;
    int keptMessages     = 0;
    int summarizedMessages = 0;
    std::string summary;        // 交接摘要（仅 L3 级非空）
    std::string detail;         // 人可读说明
};
```

```cpp
struct SummarizeRequest {
    const std::vector<Message>* messages = nullptr;   // 要被压掉的那段历史
    std::string                 previousSummary;      // 上一次的摘要（迭代压缩）
    std::string                 goal;                 // ★ 锚点原文（最初的需求）
    std::optional<std::string>  focus;                // "只关注 X"
    std::optional<std::stop_token> stop;              // ★ 摘要也要能被 Ctrl-C 打断
};
```

ISummarizer 端口

```cpp
class ISummarizer {
public:
    virtual ~ISummarizer() = default;
    virtual SummaryResult summarize(const SummarizeRequest& req) = 0;
};

struct SummarizeRequest {
    const std::vector<Message>* messages = nullptr;   // 要被压掉的那段历史
    std::string                 previousSummary;      // 上一次的摘要（迭代压缩）
    std::string                 goal;                 // ★ 锚点原文（最初的需求）
    std::optional<std::string>  focus;                // "只关注 X"
    std::optional<std::stop_token> stop;              // ★ 摘要也要能被 Ctrl-C 打断
};
```

两种实现：

实现	用途	特点
`LlmSummarizer`	真实生产路径	一次非流式请求；`temperature=0`；不写缓存；`model` 留空（K-16 修法）
`ExtractiveSummarizer`	离线 / 降级路径	本地规则抽取"交接口诀的四件事"

`ExtractiveSummarizer`摘取：原始需求 / 改过哪些文件 / 用过哪些工具 / 最近的结论

LlmSummarizer：调模型

```
1. 拼 prompt 的 user 消息内容：

   # 本任务的原始需求（**必须原样保留在摘要里**）
   把 build.sh 里的 optimize 改名为 release

   # 此前的会话摘要（请并入新摘要，不要丢信息）
   （第一次为空，第二次是上一次的摘要）

   # 需要摘要的对话
   [助手] → 调用 Read({"path": "build.sh"})
   [工具结果 call_read_0] 1      #!/bin/bash...
   [助手] → 调用 Edit({"path": "build.sh", "old_str": "optimize=1", ...})
   [工具结果 call_edit_0] 已修改 build.sh
   ...

2. 构造 ChatRequest
   system    = kCompactionPrompt（固定提示词）
   messages  = [user(上面的 body)]
   maxTokens = 2048
   temperature = 0           ← 摘要必须稳定
   cacheControl = false      ← 一次性 prompt，写缓存亏
   stop = 调用方的 stop_token ← Ctrl-C 能打断
   model = 空串               ← 让 provider 用默认值（K-16）

3. 调 provider_.chat(cr) → ChatResponse

4. 返回 SummaryResult{text, usage, note="llm:openai"}
```

```
你正在执行 CONTEXT CHECKPOINT COMPACTION（上下文检查点压缩）。
请为另一个将接手此任务的 LLM 生成交接摘要，务必包含：

1. 当前进展与已经做出的关键决策（含"为什么这么定"）
2. 重要的约束：用户偏好、项目约定、不能碰的东西
3. 所有读取过或修改过的文件的**绝对路径**（这一条最重要，一条都不能漏）
4. 还没做完的事（清晰的下一步）
5. 继续工作所必需的关键数据、示例或引用

要求：简洁、结构化、只写事实。目标是让下一个 LLM 能无缝接手，不必重读文件。
```

ExtractiveSummarizer：本地规则抽取

```
1. 扫描 history 里的所有消息，收集：
   - firstUser:     第一条 user 消息的 content
   - lastAssistant: 最后一条非空 assistant 消息的 content
   - nUser/nAsst/nTool: 各类消息的数量
   - toolUse:       map<工具名，调用次数>
   - touched:       所有工具调用参数里的"第一个短字符串参数"

2. "取第一个短字符串参数"的规则：
   for (k, v) in tc.args:
       if v 是 string 且 0 < len <= 200 且 不含 '\n':
           touched.push(v)
           break    // 每个调用只取一个

3. 拼文本：

   【本地抽取式交接摘要】已压缩 40 条消息（user=1 / assistant=20 / tool=20）
   - 更早会话的摘要（沿用以避免层层丢失）：
     （上一次摘要的前 600 字符）
   - 原始需求：把 build.sh 里的 optimize 改名为 release
   - 已用过的工具：Readx25、Editx3、Bashx2
   - 涉及的文件/参数（★ 交接口诀：路径一条都不能丢）：
     · build.sh
     · test.sh
     · ...
```

四个工具（Read/Write/Edit/Bash）的参数形状都是 {path/command: string, ...}，第一个短字符串就是路径。

```
   - 涉及的文件/参数（★ 交接口诀：路径一条都不能丢）：
     · build.sh
     · test.sh
     · ...
   - 压缩前最后的结论：（最后一条 assistant 消息的前 400 字符）

4. 截断到 maxChars（4000 字符）— 按 UTF-8 边界
```

完整例子

```
1. shouldCompact: 150000 > 10000 → 触发

2. anchorEnd = 1 (history[0] 是 user)

3. findCutPoint(history, 1, 4000):
   第一步: 从尾累加到 4000 → boundary = 80
   第二步: 往前找非 tool 消息 → 假设 history[78] 是 assistant
   → cut = 78

4. prefixEnd = 78
   老历史 = history[1..78)
   最近窗口 = history[78..103]

5. L1: cap = 10000/2 = 5000
   history[1..78) 里哪些消息 >5000 token?
   比如 history[2] 是一条 8000 token 的 tool 结果
   → fitTo 降级成 Signatures（假设降到 3000 token）

   现在总 token: 150000 - 8000 + 3000 = 145000, 还超

6. L2: foldThreshold = 1200 字符
   history[1..78) 里哪些 tool 消息 >1200 字符?
   比如 30 条 tool 结果, 每条 2500 字符
   → 每条折叠到约 400 字符
   → 节省约 30 * 2100 = 63000 字符 ≈ 15750 token

   现在总 token: 145000 - 15750 = 129250, 还超

7. L3: 调 summarizer->summarize(history[1..78))
   假设用 ExtractiveSummarizer, 输出 3000 字符 ≈ 750 token
   重建历史:
     [0] = 原 history[0] (锚点)
     [1] = user("[上下文压缩·交接摘要] ..." + 750 token 摘要)
     [2..] = 原 history[78..103] (最近窗口)

   现在总 token: 锚点 50 + 摘要 750 + 最近窗口 25000 = 25800, 还超

8. L4: 还超预算
   headTokens = est(history[0]) + est(history[1]) = 50 + 750 = 800
   剩余预算 = 10000 - 800 = 9200
   findTailStartByBudget(history, 2, 9200):
     从尾往前累加到 9200
     假设到 history[20] 时 acc = 9200 → keepFrom = 20
   删除 history[2..20)

   现在总 token: 50 + 750 + 9000 = 9800 < 10000 √

9. 抖动检查: 9800 <= 10000, attempts_ = 0
```

Anthropic 协议

构性差异

差异	OpenAI	Anthropic	谁受影响
工具结果的表达	独立一条 `role:"tool"`	`user` 消息里的 `tool_result` 块	`toMessages`
系统提示	可以放在 `messages` 里	只能在顶层 `system` 字段	`toRequest`
连续同角色消息	允许	要求严格交替（必须合并）	`toMessages` 的 `mergeInto`

StreamSink 抽象

Anthropic 是另一套事件名（`message_start` / `content_block_delta` / `input_json_delta` / `message_delta`），不可能复用 OpenAI 的 `StreamCtx`。如果 `execute()` 的签名写死 `StreamCtx*`，那么"重试 / 超时 / 错误分类 / HTTP 头"这些与协议无关的逻辑就得整份复制一遍

```cpp
struct StreamSink {
    std::function<bool(const char* data, std::size_t len)> onData;
    // 是否已经开始收到字节 —— 决定"还能不能重试"（I-8 的修法）
    std::function<bool()> started;
};
```

AnthropicStream 的事件处理

规则	OpenAI	Anthropic
一个 chunk 可能同时带多种信息	一个 chunk 同时带 content + tool_calls + finish_reason	message_delta 同时带 stop_reason + output_tokens
工具参数是增量 JSON 片段	arguments 被切成 N 段	input_json_delta 的 partial_json
只在结束时 parse 一次	finish_reason 出现时	content_block_stop 出现时

事件映射：

Anthropic 事件	对应 ChatChunk
message_start	UsageChunk（输入用量）
content_block_start（tool_use）	ToolCallStart
content_block_delta（text_delta）	TextDelta
content_block_delta（thinking_delta）	ThinkingDelta
content_block_delta（input_json_delta）	ToolCallDelta（只累积）
content_block_stop（tool_use）	ToolCallDone（parse 一次）
message_delta	UsageChunk（输出用量） + DoneChunk
error	ErrorChunk